SQL COMMANDS PRACTICAL PROJECT

Aim • Query • MySQL Terminal-Style Output • Result

Name: _____Sri varshan gandhi_____

Register Number: _____1925260276_____

Experiment 1: DDL — CREATE

Aim: To create tables in the StudentDB database.

Query:

```
USE StudentDB;
CREATE TABLE IF NOT EXISTS Department (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50) NOT NULL
);
```

Answer / Expected Result: The Department table is created.

MySQL terminal-style sample output:

```
mysql> USE StudentDB;
Database changed

mysql> CREATE TABLE IF NOT EXISTS Department (
    ->     dept_id INT PRIMARY KEY,
    ->     dept_name VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Experiment 2: DDL Constraints

Aim: To apply PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT and NOT NULL constraints.

Query:

```
CREATE TABLE IF NOT EXISTS Student (
   student_id INT PRIMARY KEY,
   student_name VARCHAR(50) NOT NULL,
   email VARCHAR(100) UNIQUE,
   age INT CHECK (age >= 16),
   marks DECIMAL(5,2) DEFAULT 0,
   dept_id INT,
   FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Answer / Expected Result: The Student table is created with the specified constraints. CHECK enforcement can vary by MySQL version.

MySQL terminal-style sample output:

```
mysql> CREATE TABLE IF NOT EXISTS Student (
    ->     student_id INT PRIMARY KEY,
    ->     student_name VARCHAR(50) NOT NULL,
    ->     email VARCHAR(100) UNIQUE,
    ->     age INT CHECK (age >= 16),
    ->     marks DECIMAL(5,2) DEFAULT 0,
    ->     dept_id INT,
    ->     FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Constraint definitions

- **PRIMARY KEY:** Uniquely identifies each record and does not allow NULL.
- **FOREIGN KEY:** Links Student.dept_id to Department.dept_id.
- **UNIQUE:** Prevents duplicate email values.
- **CHECK:** Restricts age to 16 or above.
- **DEFAULT:** Sets marks to 0 when omitted.
- **NOT NULL:** Requires student_name to have a value.

Experiment 3: DDL — ALTER

Aim: To add a new column to an existing table.

Query:

ALTER TABLE Student ADD phone VARCHAR(15);

Answer / Expected Result: The phone column is added to Student.

MySQL terminal-style sample output:

```
mysql> ALTER TABLE Student ADD phone VARCHAR(15);
Query OK, 0 rows affected (0.02 sec)
```

Experiment 4: DML — INSERT

Aim: To insert department and student records.

Query:

INSERT INTO Department VALUES (10, 'Computer Science'), (20, 'Commerce');
INSERT INTO Student (student_id, student_name, email, age, marks, dept_id) VALUES
(1, 'Anu', 'anu@example.com', 18, 92, 10),
(2, 'Bala', 'bala@example.com', 19, 78, 20),
(3, 'Charan', 'charan@example.com', 18, 85, 10);

Answer / Expected Result: Two departments and three students are inserted.

MySQL terminal-style sample output:

```
mysql> INSERT INTO Department VALUES
    ->     (10, 'Computer Science'),
    ->     (20, 'Commerce');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Student (student_id, student_name, email, age, marks, dept_id) VALUES
    ->     (1, 'Anu', 'anu@example.com', 18, 92, 10),
    ->     (2, 'Bala', 'bala@example.com', 19, 78, 20),
    ->     (3, 'Charan', 'charan@example.com', 18, 85, 10);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Experiment 5: DML — SELECT

Aim: To retrieve records from the Student table.

Query:

SELECT student_id, student_name, marks FROM Student;

Answer / Expected Result: The query displays all three student records and their marks.

MySQL terminal-style sample output:

```
mysql> SELECT student_id, student_name, marks FROM Student;
+------------+--------------+-------+
| student_id | student_name | marks |
+------------+--------------+-------+
| 1          | Anu          | 92.00 |
| 2          | Bala         | 78.00 |
| 3          | Charan       | 85.00 |
+------------+--------------+-------+
3 rows in set (0.00 sec)
```

Experiment 6: DML — UPDATE

Aim: To update marks for a selected student.

Query:

UPDATE Student SET marks = 88 WHERE student_id = 2;
SELECT student_id, student_name, marks FROM Student WHERE student_id = 2;

Answer / Expected Result: Bala's marks are updated to 88.

MySQL terminal-style sample output:

```
mysql> UPDATE Student SET marks = 88 WHERE student_id = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT student_id, student_name, marks FROM Student WHERE student_id = 2;
+------------+--------------+-------+
| student_id | student_name | marks |
+------------+--------------+-------+
| 2          | Bala         | 88.00 |
+------------+--------------+-------+
1 row in set (0.00 sec)
```

Experiment 7: DML — DELETE

Aim: To delete one student record.

Query:

DELETE FROM Student WHERE student_id = 3;
SELECT * FROM Student WHERE student_id = 3;

Answer / Expected Result: Student 3 is deleted; the verification query returns an empty set.

MySQL terminal-style sample output:

```
mysql> DELETE FROM Student WHERE student_id = 3;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Student WHERE student_id = 3;
Empty set (0.00 sec)
```

Experiment 8: TCL — SAVEPOINT and COMMIT

Aim: To save selected transaction changes and roll back to a savepoint.

Query:

```
START TRANSACTION;
UPDATE Student SET marks = 95 WHERE student_id = 1;
SAVEPOINT after_first_update;
UPDATE Student SET marks = 40 WHERE student_id = 2;
ROLLBACK TO SAVEPOINT after_first_update;
COMMIT;
```

Answer / Expected Result: The first update is committed; the second update is rolled back to the savepoint.

MySQL terminal-style sample output:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Student SET marks = 95 WHERE student_id = 1;
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT after_first_update;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Student SET marks = 40 WHERE student_id = 2;
Query OK, 1 row affected (0.00 sec)

mysql> ROLLBACK TO SAVEPOINT after_first_update;
Query OK, 0 rows affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

Experiment 9: TCL — ROLLBACK

Aim: To cancel an uncommitted update.

Query:

```
START TRANSACTION;
UPDATE Student SET marks = 50 WHERE student_id = 1;
ROLLBACK;
```

Answer / Expected Result: The uncommitted change is cancelled.

MySQL terminal-style sample output:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Student SET marks = 50 WHERE student_id = 1;
Query OK, 1 row affected (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.01 sec)
The uncommitted update was cancelled.
```

Experiment 10: DDL — TRUNCATE

Aim: To remove all rows while retaining the table structure.

Query:

```
TRUNCATE TABLE Student;
```

Answer / Expected Result: All records in Student are removed. Run this after completing other experiments.

MySQL terminal-style sample output:

```
mysql> TRUNCATE TABLE Student;
Query OK, 0 rows affected (0.02 sec)
```

Experiment 11: DDL — DROP

Aim: To remove the Student table.

Query:

DROP TABLE Student;

Answer / Expected Result: The Student table and its data are removed.

MySQL terminal-style sample output:

```
mysql> DROP TABLE Student;
Query OK, 0 rows affected (0.02 sec)
```

Result

The SQL DDL, constraint, DML and TCL commands were demonstrated with queries and terminal-style sample outputs.